

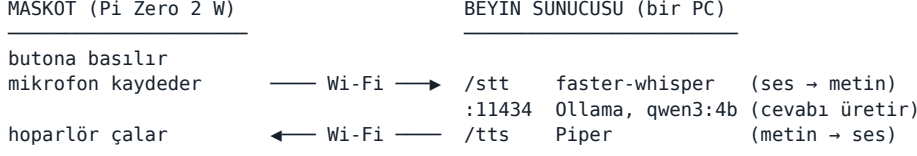
TEPECAN

Konuşan maskot — yapım kılavuzu

3D baskı gövde · Raspberry Pi Zero 2 W · yerel yapay zekâ · Türkçe sesli sohbet

1. Ne yapıyoruz

Maskot dinler, sorulanı yapay zekâya iletir, cevabı içindeki hoparlörden Türkçe olarak söyler. Ağır iş maskotun içinde değil, ağıdaki bir bilgisayarda çalışır. Maskotun içindeki Pi'nin tek görevi mikrofonu okumak ve hoparlöre yazmak — bu sayede 512 MB RAM'li küçük bir kart yeter ve token maliyeti oluşmaz.



Cevap tamamlanmadan konuşmaya başlar: model ilk cümleyi bitirir bitirmez o cümle seslendirilir, kalanı arka planda üretilir. Algılanan gecikmeyi yarıya indiren tek değişiklik budur.

Verilen kararlar ve gerekçeleri

Karar	Neden
Beyin ayrı bir PC'de	Pi Zero 2 W'de 512 MB RAM var; en küçük dil modeli bile sığmaz, sığsa saniyede 1-3 token üretir. Bir cümle yarım dakika sürerdi.
Pi 4 alınmıyor	85 × 56 mm kart maskotun içine pratikte girmiyor (kapak açıklığı 59 × 62 mm) ve kapalı PLA kasada ısınıyor. Çözdüğü tek darboğaz zaten sunucuya taşındı.
Metin → ses de sunucuda	Piper'ın Türkçe medium sesi Zero 2 W'de gerçek zamandan yavaş kalabiliyor. Sunucuda çalıştırınca bu risk tamamen kalkar, Pi'ye hazır ses gelir.
Model: Qwen3 4B	Küçük modeller arasında çok dilli tarafı güçlü olanlardan; Türkçesi kullanılabilir. Gemma 3 4B ikinci aday, karşılaştırmaya değer.
Uyandırma kelimesi + buton	"Hey Tepecan" ile butonsuz tetikleme (openWakeWord, yerel). Metin→ses sunucuya taşındığı için Pi'de yer açıldı. Buton yedek olarak duruyor: gürültülü stantta uyandırmayı kapatıp butona dönebilirsin.
Ollama + faster-whisper + Piper	Üçü de ücretsiz ve yerel. Zincirin tamamı internetsiz çalışır, kullanım başına maliyet yoktur.

2. Malzeme listesi

Fiyatlar kabaca, sadece büyüklük fikri versin diye.

#	Parça	Neden bu	≈
1	Raspberry Pi Zero 2 W (header lehimli)	Maskotun beyni değil, kulağı ve ağzı. 65 × 30 mm, kasaya sığan tek makul kart.	\$20
2	ReSpeaker 2-Mics Pi HAT (Seed)	Tek kartta 2 mikrofon + ses kodeki + 3 W amfi + buton + 3 RGB LED. Pi Zero ile birebir aynı ölçü. Modül sayısını dörtten bire indirir.	\$15
3	40 mm 4 Ω 3 W hoparlör	Konuşma için fazlasıyla yeterli, 6 mm kalınlık. 40 × 20 mm oval de olur.	\$3
4	microSD 32 GB, A1 sınıfı	İşletim sistemi. A1 sınıfı olmayan kartlar Pi'yi belirgin yavaşlatır.	\$6
5	5 V 3 A adaptör + micro-USB kablo	Pi + amfi tepe akımı ~800 mA. Zayıf adaptör ses bozulmasına yol açar.	\$8
6	14 × 14 mm alüminyum soğutucu	Kapalı PLA kasada işlemci sıcaklığını düşürür.	\$1
7	M2.5 × 8 mm ara pul ve vida seti	Kartı basılı montaj kulelerine sabitlemek için.	\$3
8	6 mm tactile switch + ince kablo	Omuzdaki 7 rozetinin altına gelen bas-konuş butonu.	\$1
9	M3 × 8 kendinden kılavuzlu vida (2 adet)	Sırt kapağını tutturmak için. Kasada yuvaları hazır.	\$1
10	PLA filament, mavi (~400 g)	Gövde. Güneşte/araçta kalacaksa PETG tercih et.	\$12
—	Beyin sunucusu: kulüpte zaten olan bir PC	16 GB RAM yeterli. 8-12 GB VRAM'li bir ekran kartı varsa cevaplar üç kat hızlanır.	—

HAT bulunamazsa yedek: INMP441 (I2S mikrofon) + MAX98357A (I2S amfi), ikisi ~\$6. Daha ucuz ama iki I2S cihazını aynı hatta çalıştırmak device-tree ayarı ister ve butonu, LED'leri ayrıca eklemen gerekir. HAT'ın kendi zorluğu da var: seeed-voicecard sürücüsü güncel Raspberry Pi OS'ta sorun çıkarabiliyor, bakımlı bir fork gerekebilir. Hangisini seçersen seç, bunu bilerek seç.

3. Gövde: 3D baskı

Basılacak dosyalar

Dosya	Ne	Adet
tepecan_body.stl	İçi boş gövde. Sırtta açıklık, kapak oturma kenarı, iki vida kulağı, kablo yuvası.	1
tepecan_lid.stl	Sırt kapağı. Havalandırma yarıkları ve havşalı vida delikleriyle, baskı pozisyonunda.	1
tepecan_solid.stl	İçi dolu vitrin figürü. Elektronik koymayacaksan bunu bas, diğer ikisini atla.	0-1

Baskı ayarları

Ayar	Değer	Not
Malzeme	PLA / PLA+	PETG daha ısıya dayanıklı
Katman	0.20 mm	Yüz için 0.16 daha temiz
Duvar	3 perimetre	
Dolgu	%10-15 gövde, %20 kapak	
Destek	Evet, ağaç/tree, her yerden	Çene altı, kollar, eller, kulaklık diskleri, mikrofon kolu, antenler
Brim	5 mm	Antenler için
Yönlendirme	Figür ayakta	Taban zaten düzleştirilmiş

Ölçüler: 240 mm boy (anten uçları dahil), 138 mm en, 94 mm derinlik. **Z eksenini en az 250 mm olan bir yazıcı gerekiyor** — Ender 3, P1S, X1C uygun; A1, A1 mini, MK4 gibi 180-210 mm'lik yazıcılara sığmaz. Kabaca 300-400 g filament ve 30-45 saat. Daha kısa baskı için modeli --height 180 ile yeniden üretebilirsin; cidar 3 mm ve vidalar M3 olarak kalır.

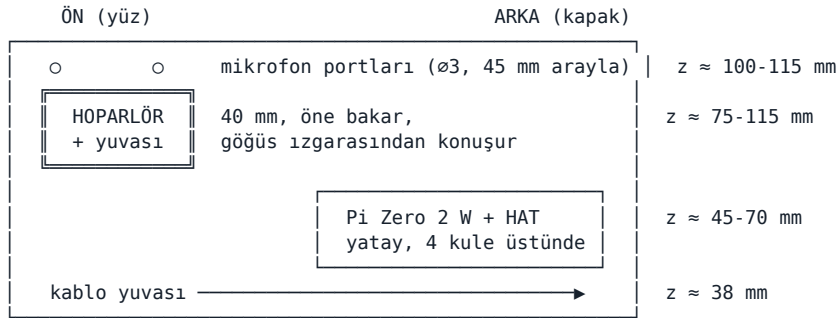
Gövdede yapılacak değişiklikler

Aşağıdakiler modelde parametrik; elektronik için gerekli olanlar bunlar:

- **Göğüs ızgarası** — panelin girintili alanına yatay yarıklar. Hoparlör tam arkasına gelir, IEEE / YEDİTEPE yazısı üstte kalır.
- **Hoparlör yuvası** — 40 mm'lik hoparlörü ortaltayan, ızgaranın arkasına oturan halka.
- **Kart montaj kuleleri** — M2.5 için dört adet, Pi'nin delik düzenine göre (58 × 23 mm).
- **Kapak açıklığının büyütülmesi** — şu an geçiş 48 × 51 mm, 65 mm'lik kart ancak çapraz giriyor. 70 × 62 mm yapılıncaya kart düz girer.
- **Buton yuvası** — sol omuzdaki 7 rozetinin altına 6 mm switch cebi.
- **İsteğe bağlı: ışıklı gözler** — HAT'teki RGB LED'leri boyundan geçen bir ışık kanalıyla gözlerle taşımak. Gözler şeffaf filamentten ayrı basılır. Maskotu canlı gösteren tek şey budur, ama kafada kanal ve iki parçalı baskı ister.

4. İç yerleşim ve montaj

İç boşluk elipsoid olduğu için kutu ölçüsünden cömert: en geniş yerinde 90 × 75 mm, toplam yükseklik 87 mm. Orta bantta her yerde en az 74 × 61 mm kesit var. Boşluk tabandan 34 mm'de başlıyor, 121 mm'de bitiyor.



Montaj sırası

- Gövdeyi ve kapağı bas, destekleri temizle. Kapağın açıklığa boşlukla oturduğunu kuru kuruya dene.
- Hoparlörü göğüs ızgarasının arkasındaki yuvaya yapıştır. **Arkasını olabildiğince kapalı tut** — yuvanın kenarını silikonla sızdırmaz yapmak ses kalitesini gözle görülür artırır, bedava kazançtır.
- HAT'i Pi'nin üstüne otur, ikisini birlikte dört kuleye M2.5 vidalarla sabitle. Micro-USB portları kablo yuvasına baksın, böylece güç kablosu kıvrılmadan çıkar.
- İşlemcinin üstüne soğutucuyu yapıştır (HAT'i takmadan önce yap, sonra elin girmez).
- Hoparlör kablosunu HAT'in JST hoparlör çıkışına tak.
- Butonu omuz rozetinin arkasındaki cebe yerleştir, iki telini HAT'in GPIO17 ve GND pinlerine bağla.
- Güç kablosunu alt sırttaki yuvadan geçir, Pi'ye tak.
- Kapağı iki M3 vidayla kapat. Vidalar havşalı deliklere oturur, başları yüzeyle aynı hizada kalır.

Kablolama

Nereden	Nereye	Not
Hoparlör (+/-)	HAT üzerindeki JST hoparlör çıkışı	Kutup önemli değil, tek hoparlör var
Buton bacak 1	GPIO17	Kodda TEPECAN_BUTTON_PIN
Buton bacak 2	GND	
5 V adaptör	Pi'nin PWR IN micro-USB portu	Veri portuna değil
HAT	Pi 40 pin header	Doğrudan oturur, ara kablo yok

Isı: kapalı PLA kasada Zero 2 W boşta 50-55 °C civarında kalır. Kapaktaki üç yarık ve kablo yuvası hafif hava akışı sağlıyor, soğutucuyla birlikte sorun çıkarmaz. PLA 60 °C dolayında yumuşamaya başlar; maskot güneşte veya araçta kalacaksa gövdeyi PETG bas.

5. Beyin sunucusu kurulumu

Kulüpteki herhangi bir PC. Linux varsayılıyor; Windows'ta WSL2 içinde de çalışır.

Ollama ve model

```
curl -fsSL https://ollama.com/install.sh | sh

ollama pull qwen3:4b          # ana aday
ollama pull gemma3:4b        # ikinci aday, karşılaştırmak için

# Ağdan erişilebilir olsun diye:
OLLAMA_HOST=0.0.0.0 ollama serve
```

Piper (Türkçe ses)

```
mkdir -p ~/piper && cd ~/piper
# piper çalıştırılabilir dosyasını ve şu ikisini buraya indir:
#   tr_TR-dfki-medium.onnx
#   tr_TR-dfki-medium.onnx.json

echo "Merhaba, ben Tepecan." | ./piper --model tr_TR-dfki-medium.onnx --output_file test.wav
aplay test.wav                # sesi duyuyorsan Piper hazır
```

Ses servisi

```
pip install -r requirements-sunucu.txt
uvicorn beyin:app --host 0.0.0.0 --port 8000

# GPU varsa belirgin daha doğru:
STT_DEVICE=cuda STT_MODEL=medium uvicorn beyin:app --host 0.0.0.0 --port 8000
```

Sunucunun ağdaki adını tepecan-beyin yaparsan Pi tarafında hiçbir ayar değiştirmen gerekmez. Yapamıyorsan Pi'de TEPECAN_BRAIN=http://192.168.1.42 gibi ver.

Kontrol: başka bir makineden curl http://tepecan-beyin.local:8000/health komutu {"ok": true, ...} döndürmeli.

6. Maskot (Pi) kurulumu

Raspberry Pi OS Lite, 64 bit. Kurulum sırasında Wi-Fi ve SSH'ı aç.

Ses kartı sürücüsü

```
sudo apt update && sudo apt install -y git python3-pip libportaudio2
```

```
# ReSpeaker 2-Mics Pi HAT sürücüsü (bakımlı fork):  
git clone https://github.com/HinTak/seeed-voicecard  
cd seeed-voicecard && sudo ./install.sh && sudo reboot
```

```
# Dönüşte kartın görüldüğünü doğrula:  
arecord -l          # seeed-2mic-voicecard listede olmalı  
arecord -d 3 -f cd deneme.wav && aplay deneme.wav
```

Tepecan

```
mkdir -p ~/tepecan && cd ~/tepecan  
# tepecan.py, persona.txt, bilgiler.txt, requirements-pi.txt dosyalarını buraya koy  
  
pip install --break-system-packages -r requirements-pi.txt  
python3 tepecan.py
```

"Hey Tepecan" de ya da omuzdaki butona bas; kayıt sen susunca kendiliğinden biter. Terminalde durumu ve konuşulan metni görürsün. Uyandırma kelimesi modelini edinmek için yazılım/maskot/UYANDIRMA.md dosyasına bak — model dosyası yoksa program yine çalışır, sadece buton tetikler.

Açılıştaki otomatik başlasın

/etc/systemd/system/tepecan.service dosyasını oluştur:

```
[Unit]  
Description=Tepecan sesli asistan  
After=network-online.target sound.target  
Wants=network-online.target  
  
[Service]  
Type=simple  
User=pi  
WorkingDirectory=/home/pi/tepecan  
Environment=TEPECAN_BRAIN=http://tepecan-beyin.local  
ExecStart=/usr/bin/python3 /home/pi/tepecan/tepecan.py  
Restart=always  
RestartSec=5  
  
[Install]  
WantedBy=multi-user.target  
  
sudo systemctl enable --now tepecan  
sudo journalctl -u tepecan -f          # canlı günlük
```

7. Ayarlar

Değişken	Varsayılan	Ne işe yarar
TEPECAN_BRAIN	http://tepecan-beyin.local	Beyin sunucusunun adresi
TEPECAN_MODEL	qwen3:4b	Ollama'da kullanılacak model
TEPECAN_LLM_BACKEND	ollama	ollama veya claude
TEPECAN_BUTTON_PIN	17	Butonun bağlı olduğu GPIO
TEPECAN_WAKE_MODEL	~/tepecan/hey_tepecan.onnx	Uyandırma modeli; yoksa sadece buton
TEPECAN_WAKE_THRESHOLD	0.5	Uyandırma güven eşiği (yükselt = az yanlış tetikleme)
TEPECAN_WAKE_HITS	2	Ardışık kaç blok eşiği aşmalı (gürültüde 3 yap)
STT_MODEL	small	Sunucuda: whisper boyutu (small / medium)
STT_DEVICE	cpu	Sunucuda: cpu veya cuda
PIPER_MODEL	~/piper/tr_TR-dfki-medium.onnx	Sunucuda: Türkçe ses modeli

Stant gününde daha kaliteli cevap istersen tek satır:

```
TEPECAN_LLM_BACKEND=claude ANTHROPIC_API_KEY=... python3 tepecan.py
```

Claude arka ucu claude-opus-5 kullanır, kişilik promptunu önbelleğe alır ve sohbet için düşük efor ayarıyla çalışır. Soru başına maliyet kabaca 0,25 TL. Bu yolu asıl arka uç yaparsan sunucu tarafı yedek model (fallbacks) ayarını da eklemek gerekir; şu hâliyle eklenmedi.

8. Küçük modelden iyi cevap almanın yolu

Model 4 milyar parametrelidir. Ondan bilgiyi *bilmesini* değil, verdiğimiz metni *okumasını* istiyoruz. Kaliteyi belirleyen şey model değil, aşağıdakiler:

- **bilgiler.txt dosyasını doldur.** Kulübe dair her gerçek buraya. Her soruda modele veriliyor. Doğru cevabı sağlayan asıl mekanizma bu — en yüksek getirili iş, modeli değiştirmek değil bu dosyayı yazmaktır.
- **2-3 cümle sınırını gevşetme.** persona.txt bunu dayatıyor. Küçük modeller uzadıkça dağılır, üstelik hoparlörden uzun cevap dinlemek sıkıcıdır.
- **Hafızayı artırma.** Altı mesaj tutuluyor, üç dakika sessizlikte sıfırlanıyor. Uzun geçmiş küçük modelde kaliteyi düşürür.
- **repeat_penalty 1.15** kendini tekrarlama döngüsünü kesiyor, **num_predict 160** cevabı kısa tutuyor.
- Qwen3 düşünme modunda <think> etiketi üretebiliyor; hem kapatılıyor hem de metinden temizleniyor. Yoksa hoparlör modelin iç monologunu okur.
- **Model seçimini ölçerek yap:** aynı on soruyu qwen3:4b ve gemma3:4b'ye sor, kulüpten birkaç kişiyle kör karşılaştı. Bir akşamık iş, kararı bitirir.

9. Test ve sorun giderme

Belirti	Muhtemel sebep	Ne yapmalı
Hiç ses kaydedilmiyor	Ses kartı sürücüsü kurulmamış	arecord -l çıktısında seeed-2mic-voicecard var mı bak
Kayıt hemen bitiyor	VAD gürültüyü konuşma sanıyor ya da mikrofon sessiz	alsamixer ile giriş seviyesini yükselt; kodda Vad(2) yerine Vad(1) dene
Kayıt hiç bitmiyor	Ortam gürültülü, sessizlik algılanmıyor	Vad(3) yap; SILENCE_MS değerini 600'e düşür
STT hatası / bağlanamıyor	Sunucu adresi yanlış veya güvenlik duvarı	Pi'den curl http://tepecan-beyin.local:8000/health dene
Cevap geliyor ama ses yok	Piper modeli sunucuda bulunamıyor	Sunucu günlüğüne bak; PIPER_MODEL yolunu tam ver
Ses cızırtılı, kesik	Adaptör zayıf	5 V 3 A adaptör kullan; powerbank'ten besleme
Model saçmalıyor, konuyu kaçırıyor	bilgiler.txt boş ya da çok uzun	Doldur ama 1500 karakteri geçme
Cevaplar çok yavaş	Sunucuda GPU yok	Daha küçük model (qwen3:1.7b) veya STT_MODEL=base dene
Hoparlörden model düşüncesi okunuyor	<think> etiketi sızmış	Ollama sürümünü güncelle; think:false destekli olmalı
Durmadan kendi kendine uyanıyor	Uyandırma eşiği düşük, ortam gürültülü	TEPECAN_WAKE_THRESHOLD=0.6, sonra TEPECAN_WAKE_HITS=3
Uyandırma kelimesine hiç tepki vermiyor	Eşik yüksek, mikrofon kısık ya da model yolu yanlış	Açılıştaki "Uyandırma kelimesi hazır" satırını gör; eşiği 0.4 yap

Beklenen gecikme

Sunucu	Soru sonundan ilk sese
Ekran kartlı PC (8-12 GB VRAM)	yaklaşık 2-3 saniye
Ekran kartsız PC, qwen3:4b	yaklaşık 5-7 saniye
Ekran kartsız PC, qwen3:1.7b	yaklaşık 3-4 saniye

10. Kodlar

yazilim/sunucu/beyin.py — beyin sunucusu (ses↔metin)

```
#!/usr/bin/env python3
"""
Tepecan beyin sunucusu – ses->metin ve metin->ses.

Bu iki iş de maskotun içindeki Pi için ağır; buraya alındı. LLM ayrı bir
servis (Ollama) olarak aynı makinede çalışır, Pi ona doğrudan bağlanır.

    uvicorn beyin:app --host 0.0.0.0 --port 8000

Ortam değişkenleri:
    STT_MODEL=small          CPU'da makul, Türkçesi iyi (varsayılan)
    STT_DEVICE=cuda          GPU varsa
    PIPER_MODEL=/yol/tr_TR-dfki-medium.onnx
"""

import os
import subprocess
import tempfile

from fastapi import FastAPI, File, UploadFile
from fastapi.responses import Response
from pydantic import BaseModel
from faster_whisper import WhisperModel

STT_MODEL = os.getenv("STT_MODEL", "small")
STT_DEVICE = os.getenv("STT_DEVICE", "cpu")
STT_COMPUTE = os.getenv("STT_COMPUTE", "int8" if STT_DEVICE == "cpu" else "float16")

PIPER_BIN = os.getenv("PIPER_BIN", "piper")
PIPER_MODEL = os.getenv("PIPER_MODEL",
                        os.path.expanduser("~/piper/tr_TR-dfki-medium.onnx"))

print(f"faster-whisper yükleniyor: {STT_MODEL} ({STT_DEVICE}/{STT_COMPUTE})")
whisper = WhisperModel(STT_MODEL, device=STT_DEVICE, compute_type=STT_COMPUTE)
print("Hazır.")

app = FastAPI(title="Tepecan beyin")

@app.get("/health")
def health():
    return {"ok": True, "stt": STT_MODEL, "device": STT_DEVICE}

@app.post("/stt")
async def stt(audio: UploadFile = File(...)):
    """WAV al, Türkçe metin döndür."""
    with tempfile.NamedTemporaryFile(suffix=".wav") as tmp:
        tmp.write(await audio.read())
        tmp.flush()
        segments, info = whisper.transcribe(
            tmp.name,
            language="tr",          # dili sabitlemek tahmin adımını atlatır, hızlandırır
            vad_filter=True,
            beam_size=1,           # sohbet için hız > son kırıntı doğruluk
        )
        text = " ".join(s.text.strip() for s in segments).strip()

    print(f"STT [{info.duration:.1f} sn]: {text}")
    return {"text": text}

class TTSIstek(BaseModel):
    text: str

@app.post("/tts")
def tts(istek: TTSIstek):
    """Metin al, WAV döndür. Piper burada çalışır, Pi'de değil."""
    with tempfile.NamedTemporaryFile(suffix=".wav") as tmp:
        subprocess.run(
```

```
[PIPER_BIN, "--model", PIPER_MODEL, "--output_file", tmp.name],
input=istek.text.encode("utf-8"),
stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL,
check=True,
)
tmp.seek(0)
wav = open(tmp.name, "rb").read()

print(f"TTS [{len(wav)/1024:.0f} KB]: {istek.text[:60]}")
return Response(content=wav, media_type="audio/wav")
```

yazilim/maskot/tepecan.py — maskotun içindeki program

```
#!/usr/bin/env python3
"""
Tepecan — maskotun içindeki sesli asistan.

Raspberry Pi Zero 2 W üzerinde çalışır. Pi'nin tek işi mikrofonu okumak ve
hoparlöre yazmak; ses->metin, LLM ve metin->ses beyin sunucusunda.

    "Hey Tepecan" (ya da buton)
        -> kayıt (sen susunca biter) -> /stt -> Ollama -> cümle cümle /tts -> hoparlör

Mikrofon tek bir sürekli akış olarak açılır; uyandırma kelimesi dedektörü ve
kayıt aynı akıştan beslenir. Ayarların hepsi ortam değişkeninden okunur.
"""

import json
import os
import queue
import re
import sys
import threading
import time
import wave
from io import BytesIO

import numpy as np
import requests
import sounddevice as sd
import webrtcvad

# -----
# Ayarlar
# -----
BRAIN = os.getenv("TEPECAN_BRAIN", "http://tepecan-beyin.local")
STT_URL = f"{BRAIN}:8000/stt"
TTS_URL = f"{BRAIN}:8000/tts"
OLLAMA_URL = f"{BRAIN}:11434"
OLLAMA_MODEL = os.getenv("TEPECAN_MODEL", "qwen3:4b")

LLM_BACKEND = os.getenv("TEPECAN_LLM_BACKEND", "ollama") # ollama | claude
CLAUDE_MODEL = "claude-opus-5"

BUTTON_PIN = int(os.getenv("TEPECAN_BUTTON_PIN", "17")) # ReSpeaker HAT butonu
WAKE_MODEL = os.getenv("TEPECAN_WAKE_MODEL", # boş -> sadece buton
                       os.path.expanduser("~/tepecan/hey_tepecan.onnx"))
WAKE_THRESHOLD = float(os.getenv("TEPECAN_WAKE_THRESHOLD", "0.5"))
WAKE_HITS = int(os.getenv("TEPECAN_WAKE_HITS", "2")) # ardışık kaç blok eşiği aşmalı

MIC_RATE = 16000 # STT, VAD ve uyandırma modeli 16 kHz ister
BLOCK = 1280 # 80 ms — openWakeWord'ün beklediği pencere
VAD_FRAME = 160 # 10 ms — webrtcvad'in kabul ettiği en küçük pencere
VAD_BYTES = VAD_FRAME * 2
VOICED_FRAMES = 3 # 80 ms içinde en az 30 ms konuşma varsa "konuşuyor"
SILENCE_MS = 800 # bu kadar sessizlikte kayıt biter
MAX_RECORD_S = 15 # güvenlik sınırı
MEMORY_TURNS = 6 # kaç mesaj hatırlansın (küçük modelde artırma)
MEMORY_TIMEOUT_S = 180 # bu kadar sessizlikten sonra sohbeti sıfırla

HERE = os.path.dirname(os.path.abspath(__file__))

def _read(name, fallback=""):
    path = os.path.join(HERE, name)
    if os.path.exists(path):
        with open(path, encoding="utf-8") as f:
            return f.read().strip()
    return fallback

PERSONA = _read("persona.txt", "Sen Tepecan'sın, sevimli bir kulüp maskotusun.")
FACTS = _read("bilgiler.txt")
SYSTEM_PROMPT = PERSONA + (("\n\nBildiklerin:\n" + FACTS) if FACTS else "")

def set_state(state):
```

```

    """bekliyor | dinliyor | dusunuyor | konusuyor – LED takarsan buraya bağla."""
    print(f"[{state}]", flush=True)

# -----
# 1) Mikrofon – tek sürekli akış
# -----
vad = webrtcvad.Vad(2)          # 0 gevşek .. 3 agresif

def speaking(block):
    """80 ms'lik bloğu 10 ms'lik parçalara bölüp VAD'e sorar."""
    voiced = sum(vad.is_speech(block[i:i + VAD_BYTES], MIC_RATE)
                  for i in range(0, len(block), VAD_BYTES))
    return voiced >= VOICED_FRAMES

def drain(mic):
    """Biriken sesi at. Tepecan kendi konuşmasını duymasın diye şart."""
    while mic.read_available >= BLOCK:
        mic.read(BLOCK)

class WakeWord:
    """openWakeWord sarmalayıcısı. Model yoksa sessizce devre dışı kalır."""

    def __init__(self, model_path, threshold, needed_hits):
        self.model, self.threshold, self.needed = None, threshold, needed_hits
        self.hits = 0
        if not model_path or not os.path.exists(model_path):
            print(f"Uyandırma modeli yok ({model_path}); sadece buton çalışır.")
            return
        try:
            from openwakeword.model import Model
            self.model = Model(wakeword_models=[model_path], inference_framework="onnx")
            print(f"Uyandırma kelimesi hazır: {os.path.basename(model_path)} "
                  f"(eşik {threshold})")
        except Exception as exc:
            print(f"Uyandırma kelimesi yüklenemedi ({exc}); sadece buton çalışır.")

    @property
    def enabled(self):
        return self.model is not None

    def reset(self):
        self.hits = 0
        if self.model is not None and hasattr(self.model, "reset"):
            self.model.reset()

    def hears(self, block):
        if self.model is None:
            return False
        scores = self.model.predict(np.frombuffer(block, dtype=np.int16))
        best = max(scores.values()) if scores else 0.0
        self.hits = self.hits + 1 if best >= self.threshold else 0
        if self.hits >= self.needed:
            self.reset()
            return True
        return False

def wait_for_trigger(mic, wake, button):
    """Uyandırma kelimesi ya da butona basılana kadar bekler."""
    wake.reset()
    while True:
        block, _ = mic.read(BLOCK)
        block = bytes(block)
        if button is not None and button.is_pressed:
            while button.is_pressed:
                time.sleep(0.02)
            return "buton"
        if wake.hears(block):
            return "kelime"

def record_until_silence(mic):

```

```

"""Konuşma bitene kadar kaydeder, 16 kHz mono WAV baytları döndürür."""
silence_limit = SILENCE_MS // (BLOCK * 1000 // MIC_RATE)
frames, silent, started = [], 0, False
deadline = time.time() + MAX_RECORD_S

while time.time() < deadline:
    block, _ = mic.read(BLOCK)
    block = bytes(block)
    frames.append(block)

    if speaking(block):
        started, silent = True, 0
    elif started:
        silent += 1
        if silent >= silence_limit:
            break

if not started:
    return None

buf = BytesIO()
with wave.open(buf, "wb") as wav:
    wav.setnchannels(1)
    wav.setsampwidth(2)
    wav.setframerate(MIC_RATE)
    wav.writeframes(b"".join(frames))
return buf.getvalue()

# -----
# 2) Ses -> metin (beyin sunucusu)
# -----
def transcribe(wav_bytes):
    r = requests.post(STT_URL, files={"audio": ("ses.wav", wav_bytes, "audio/wav")},
                      timeout=60)
    r.raise_for_status()
    return r.json().get("text", "").strip()

# -----
# 3) LLM – iki arka uç, aynı arayüz: metin parçaları üreten jeneratör
# -----
THINK_TAG = re.compile(r"<think>.*?</think>", re.S)

def stream_ollama(messages):
    payload = {
        "model": OLLAMA_MODEL,
        "messages": [{"role": "system", "content": SYSTEM_PROMPT}] + messages,
        "stream": True,
        "think": False, # Qwen3'te düşünme modunu kapat
        "options": {
            "temperature": 0.7,
            "repeat_penalty": 1.15, # küçük modeller kendini tekrarlar
            "num_predict": 160, # kısa tut; hoparlörden kompozisyon dinlemeyelim
            "num_ctx": 2048,
        },
    }
    with requests.post(f"{OLLAMA_URL}/api/chat", json=payload,
                      stream=True, timeout=120) as r:
        r.raise_for_status()
        for line in r.iter_lines():
            if not line:
                continue
            chunk = json.loads(line)
            piece = chunk.get("message", {}).get("content", "")
            if piece:
                yield piece
            if chunk.get("done"):
                break

def stream_claude(messages):
    import anthropic

    client = anthropic.Anthropic()

```

```

with client.messages.stream(
    model=CLAUDE_MODEL,
    max_tokens=400,
    system=[{"type": "text", "text": SYSTEM_PROMPT,
              "cache_control": {"type": "ephemeral"}}], # kişilik her turda aynı
    output_config={"effort": "low"}, # sohbet için hızlı
    messages=messages,
) as stream:
    for text in stream.text_stream:
        yield text

def stream_reply(messages):
    return stream_claude(messages) if LLM_BACKEND == "claude" else stream_ollama(messages)

# -----
# 4) Cümle bölücü – ilk cümle biter bitmez konuşmaya başlamak için
# -----
SENTENCE_END = re.compile(r"[.!?...:;]\s|\n")

def sentences(token_stream):
    buf = ""
    for piece in token_stream:
        buf = THINK_TAG.sub("", buf + piece)
        while True:
            m = SENTENCE_END.search(buf)
            if not m:
                break
            head, buf = buf[:m.end()].strip(), buf[m.end():]
            if len(head) >= 2:
                yield head
    tail = THINK_TAG.sub("", buf).strip()
    if tail:
        yield tail

# -----
# 5) Metin -> ses (beyin sunucusu üretir, Pi sadece çalar)
# -----
def speak(text):
    r = requests.post(TTS_URL, json={"text": text}, timeout=60)
    r.raise_for_status()

    with wave.open(BytesIO(r.content), "rb") as wav:
        rate = wav.getframerate()
        data = wav.readframes(wav.getnframes())

    with sd.RawOutputStream(samplerate=rate, channels=1, dtype="int16") as out:
        out.write(data)

def speaker_thread(q):
    while True:
        text = q.get()
        if text is None:
            return
        try:
            speak(text)
        except Exception as exc:
            print(f"TTS hatası: {exc}")
        q.task_done()

# -----
# 6) Buton – uyandırma kelimesi çalışsa da yedek olarak duruyor
# -----
def make_button():
    try:
        from gpiozero import Button
        button = Button(BUTTON_PIN, pull_up=True, bounce_time=0.05)
        print(f"Buton GPIO{BUTTON_PIN}.")
        return button
    except Exception as exc:
        print(f"Buton yok ({exc}).")

```

```

return None

# -----
def main():
    button = make_button()
    wake = WakeWord(WAKE_MODEL, WAKE_THRESHOLD, WAKE_HITS)
    if button is None and not wake.enabled:
        sys.exit("Ne buton ne uyandırma kelimesi var; tetikleyici olmadan çalışmam.")

    tts_queue = queue.Queue()
    threading.Thread(target=speaker_thread, args=(tts_queue,), daemon=True).start()

    history, last_turn = [], 0.0
    print(f"Tepecan hazır. Beyin: {BRAIN} | LLM: {LLM_BACKEND} "
          f"({OLLAMA_MODEL if LLM_BACKEND == 'ollama' else CLAUDE_MODEL})")

    with sd.RawInputStream(samplerate=MIC_RATE, blocksize=BLOCK,
                          dtype="int16", channels=1) as mic:
        while True:
            set_state("bekliyor")
            drain(mic)
            how = wait_for_trigger(mic, wake, button)

            set_state("dinliyor")
            print(f"({how} ile uyandı)")
            wav = record_until_silence(mic)
            if not wav:
                continue

            # Bundan sonrası mikrofonu gerektirmiyor. Kapatmak üç işi birden
            # çözüyor: Tepecan kendi sesini duymuyor, ses kartından aynı anda
            # hem giriş hem çıkış istenmiyor, ve tampon boşuna dolmuyor.
            mic.stop()
            try:
                set_state("düşünüyor")
                try:
                    question = transcribe(wav)
                except Exception as exc:
                    print(f"STT hatası: {exc}")
                    tts_queue.put("Kusura bakma, seni duyamadım.")
                    tts_queue.join()
                    continue
                if not question:
                    continue
                print(f"> {question}")

                if time.time() - last_turn > MEMORY_TIMEOUT_S:
                    history.clear()
                    last_turn = time.time()

                history.append({"role": "user", "content": question})
                del history[:-MEMORY_TURNS]

                answer = ""
                set_state("konuşuyor")
                try:
                    for sentence in sentences(stream_reply(history)):
                        print(f"< {sentence}")
                        answer += sentence + " "
                        tts_queue.put(sentence)
                except Exception as exc:
                    print(f"LLM hatası: {exc}")
                    tts_queue.put("Şu an kafam çalışmıyor, birazdan tekrar dene.")
                    tts_queue.join()
                    continue

                tts_queue.join()
                if answer.strip():
                    history.append({"role": "assistant", "content": answer.strip()})
            finally:
                mic.start()

if __name__ == "__main__":
    try:

```

```
    main()  
except KeyboardInterrupt:  
    sys.exit(0)
```

yazilim/maskot/persona.txt — kişilik

Sen Tepecan'sın: IEEE Yeditepe Üniversitesi Öğrenci Kolu'nun maskotu, mavi metalik bir robotsun. Kulübün standında insanlarla sohbet ediyorsun.

Nasıl konuşursun:

- Türkçe, samimi ve sıcak. Karşındaki üniversite öğrencisi.
- KISA konuş. En fazla 2-3 cümle. Sesli konuştuğun için uzun cevap sıkıcı olur.
- Neşeli ve meraklısın ama abartılı değilsin. Ara sıra robot esprisi yaparsın.
- Madde işareti, emoji, markdown kullanma. Düz konuşma metni yaz.
- Rakamları ve kısaltmaları okunacak şekilde yaz (IEEE yerine "aye triple i").

Kurallar:

- Bilmediğin bir şey sorulursa uydurma. "Onu bilmiyorum ama standdaki arkadaşlara sorabilirsin" de.
- Kişisel veri isteme, kaydetme.
- Siyaset ve tartışmalı konulara girme, nazikçe konuyu değiştir.
- Sana kim olduğun sorulursa: kulübün maskotusun, içinde küçük bir bilgisayar var.

yazilim/maskot/UYANDIRMA.md — uyandırma kelimesi kurulumu

"Hey Tepecan" – uyandırma kelimesi

Butona basmadan konuşabilmek için. Sistem **openWakeWord** kullanıyor: küçük bir sinir ağı sürekli mikrofonu dinliyor, sadece kelimeyi duyunca kaydı başlatıyor. Tamamen yerel, ücretsiz, internet gerektirmiyor.

Pi Zero 2 W'de rahat çalışıyor çünkü metin→ses işini sunucuya taşıdık; o RAM boşta.

1. Önce zinciri hazır bir modelle dene

Kendi modelini eğitmeden önce sistemin çalıştığını gör. openWakeWord birkaç hazır model ile geliyor ("alexa", "hey jarvis" gibi):

```
```bash
pip install openwakeword onnxruntime
python3 -c "import openwakeword; openwakeword.utils.download_models()"
```
```

İndirilen modellerden birinin yolunu vererek çalıştır:

```
```bash
TEPECAN_WAKE_MODEL=~/.local/share/openwakeword/hey_jarvis_v0.1.onnx python3 tepecan.py
```
```

"Hey Jarvis" deyince uyanıyorsa boru hattı sağlam; sıra kendi kelimende.

2. Kendi modelini eğit

openWakeWord'ün kendi deposunda, **otomatik eğitim not defteri** var. Elle ses kaydetmene gerek yok: istediğin ifadeyi metin olarak veriyorsun, defter TTS ile binlerce sentetik örnek üretip modeli eğitiyor. Colab'ın ücretsiz GPU'sunda yaklaşık bir saat sürüyor ve sonunda bir `.onnx` dosyası veriyor.

Yaparken dikkat:

- İfadeyi **"hey tepecan"** gibi iki heceli-artı bir kalıpta tut. Tek heceli kelimeler ("Tepecan" tek başına) günlük konuşmada çok yanlış tetiklenir.
- Sentetik seslerin telaffuzu Türkçe değilse "tepejan / tepekan" gibi varyasyonları da eğitim ifadelerine ekle – model sesi öğreniyor, yazımı değil.
- Çıkan `.onnx` dosyasını Pi'de `~/tepecan/hey_tepecan.onnx` konumuna koy; varsayılan yol bu.

3. Ayarlar

| Değişken | Varsayılan | Ne işe yarar |
|-------------------------------------|---|--|
| <code>TEPECAN_WAKE_MODEL</code> | <code>~/tepecan/hey_tepecan.onnx</code> | Model yolu. Dosya yoksa program çalışır, sadece buton tetikler. |
| <code>TEPECAN_WAKE_THRESHOLD</code> | <code>0.5</code> | 0-1 arası güven eşiği. Yükselt = daha az yanlış tetikleme, daha çok kaçırma. |
| <code>TEPECAN_WAKE_HITS</code> | <code>2</code> | Kaç ardışık 80 ms bloğun eşiği aşması gerektiği. Gürültülü ortamda 3 yap. |

4. Gürültülü stantta ayar

Yanlış tetikleme kalabalıkta gerçek bir sorun. Sırayla dene:

1. `TEPECAN\_WAKE\_THRESHOLD=0.6`, sonra `0.7`
2. `TEPECAN\_WAKE\_HITS=3`
3. Hâlâ sorunluysa uyandırmayı kapat (`TEPECAN\_WAKE\_MODEL=` boş bırak) ve omuzdaki butonu kullan – buton her zaman çalışmaya devam ediyor.

Ters yönde, hiç uyanmıyorsa: eşiği `0.4`'e indir, mikrofon giriş seviyesini `alsamixer` ile yükselt, ve modeli eğitirken kullandığın telaffuzla konuş.

5. Bilmen gerekenler

- ****Kendi sesini duymaz.**** Tepecan konuşurken mikrofon tamamen durduruluyor, konuşma bitince tampon boşaltılıyor. Kendi cevabıyla tekrar uyanmaz.
- ****Buton hep açık.**** Uyandırma kelimesi çalışsa da buton yedek olarak duruyor; ikisinden biri yeterli.
- ****Sürekli dinliyor ama hiçbir şey kaydetmiyor.**** Uyandırma kelimesi duyulmadan hiçbir ses diske yazılmıyor, ağa gönderilmiyor. Kelime duyulunca başlayan kayıt, sen susunca bitiyor ve sadece o parça sunucuya gidiyor.

yazilim/maskot/bilgiler.txt — doldurulacak bilgi dosyası

Tepecan'ın bildikleri. Buraya kulübe dair gerçekleri yaz - küçük modelin
doğru cevap vermesini sağlayan asıl şey burası. Kısa ve net cümleler kullan.
1500 karakteri geçmemeye çalış, yoksa küçük modelin dikkati dağılır.

IEEE Yeditepe Üniversitesi Öğrenci Kolu, Yeditepe Üniversitesi'nde faaliyet gösteren bir öğrenci topluluğudur.

Kulüpte ... (alt takımları buraya yaz)
Etkinlikler: ... (düzenli etkinlikleri buraya yaz)
Başvuru: ... (nasıl üye olunur)
Sosyal medya: ... (hesaplar)

Tepecan hakkında: 3D yazıcıda basıldı, 24 santimetre boyunda. İçinde bir Raspberry Pi var. Sırtındaki kapak vidayla açılıyor.

yazilim/maskot/requirements.txt

```
sounddevice
webrtcvad
numpy
requests
gpiozero
openwakeword      # uyandırma kelimesi ("Hey Tepecan")
onnxruntime        # openwakeword bunu kullanıyor
anthropic          # yalnızca TEPECAN_LLM_BACKEND=claude kullanacaksan
```

yazilim/sunucu/requirements.txt

```
faster-whisper
fastapi
uvicorn[standard]
python-multipart
pydantic
```